



中山大學

SUN YAT-SEN UNIVERSITY

多模态课程大作业

基于 VLM 的智能图文问答助手——系统实现、评测与分析

姓 名 廖展辉，洪衍锴，曾睿

学 号

学 院 计算机学院

专 业 计算机科学与技术

2026 年 6 月 16 日

摘 要

随着视觉语言模型 (Vision-Language Model, VLM) 的快速发展, 多模态图文理解技术日趋成熟。本项目设计并实现了一个基于 VLM 的智能图文问答助手——vlm-chat, 支持用户上传图片和文档并进行中文多轮问答交互。系统以阿里云 DashScope API 提供的 Qwen3-VL-Flash 模型为核心推理引擎, 实现了流式输出、多 Provider 热切换、RAG 知识库检索、DuckDuckGo 联网工具调用等工程特性, 并构建了 Gradio + React TypeScript 双前端交互界面。

为全面评估系统能力, 本项目构建了包含 80 条中文数据的自建评测集 (覆盖自然场景与文档图像两大类、五子类别), 并在 TextVQA 0.5.1 公开数据集上进行了验证。修复评测算法中的虚高问题后, 自建集准确率为 55.0%, 公开集 (TextVQA) 准确率为 86.1%。通过 5 个成功案例和 5 个失败案例的深度分析, 揭示了模型在物体识别召回偏差、否定回答场景、中文 OCR 细粒度等方面的典型能力边界。基于实验结果, 从 AGI 视角讨论了多模态理解的当前局限与未来方向, 提出了 Sentence-BERT 语义检索、LoRA 微调和视频问答扩展三条改进路径。

关键词: 视觉语言模型, 多模态问答, 智能助手, VLM 评测, TextVQA

摘 要

With the rapid development of Vision-Language Models (VLMs), multimodal image-text understanding technology has become increasingly mature. This project designs and implements vlm-chat, an intelligent image-text QA assistant based on VLM, supporting image and document upload for Chinese multi-turn QA interactions. The system uses the Qwen3-VL-Flash model provided by Alibaba Cloud DashScope API as the core inference engine, implementing engineering features including streaming output, multi-provider hot-swap, RAG knowledge base retrieval, and DuckDuckGo web search tool calling, with Gradio and React TypeScript dual frontend interfaces.

To comprehensively evaluate system capabilities, we constructed a self-built evaluation dataset of 80 Chinese items covering natural scenes and document images across five subcategories, and validated on the TextVQA 0.5.1 public benchmark. After fixing inflation issues in the evaluation algorithm, the self-built dataset achieves 55.0% accuracy while the public dataset (TextVQA) achieves 86.1%. Through in-depth analysis of five success and five failure cases, we reveal typical capability boundaries in object recognition recall bias, negation scenarios, and fine-grained Chinese OCR. Based on experimental results, we discuss current limitations and future directions of multimodal understanding from an AGI perspective, proposing three improvement paths: Sentence-BERT semantic retrieval, LoRA fine-tuning, and video QA extension.

Key Words: Vision-Language Model, Multimodal QA, Intelligent Assistant, VLM Evaluation, TextVQA

目录

1 系统实现	1
1.1 系统架构	1
1.1.1 各层职责	2
1.2 核心模块设计	2
1.2.1 多 Provider API 封装 (api_client.py)	2
1.2.2 会话管理 (chat_manager.py)	2
1.2.3 图片与文档处理 (image_processor.py)	3
1.2.4 RAG 知识库 (rag.py)	3
1.2.5 工具调用 (tools.py)	4
1.2.6 向量嵌入 (embeddings.py)	4
1.3 系统亮点	4
1.3.1 流式输出 (Streaming)	4
1.3.2 多 Provider 热切换	4
1.3.3 RAG 知识库检索	5
1.3.4 DuckDuckGo 联网工具调用	5
1.3.5 限流与生产鉴权	5
1.3.6 双前端架构	5
1.4 技术栈总览	6
2 实验结果	7
2.1 实验设置	7
2.1.1 自建评测集	7
2.1.2 公开评测集	7
2.1.3 评测方法	7
2.2 自建集评测结果	8
2.2.1 总体结果	8
2.2.2 分类结果	8
2.2.3 结果分析	8
2.3 公开集评测结果	9
2.3.1 总体结果	9

2.4 对比分析	9
2.4.1 自建集与公开集对比	9
2.4.2 可视化分析	10
2.5 UI 展示及实际运行效果	11
2.5.1 讲义截图理解	11
2.5.2 多轮对话上下文保持	12
2.6 小结	12
3 案例分析	13
3.1 成功案例分析	13
3.2 失败案例分析	13
3.2.1 否定回答与期望矛盾（案例一至四）	13
3.2.2 细粒度视觉识别不足（案例五）	14
3.3 失败模式总结	14
4 反思与展望	15
4.1 模型能力边界反思	15
4.1.1 物体识别的召回偏差	15
4.1.2 中文 OCR 的边界	15
4.1.3 摘要总结的深层语义理解不足	15
4.2 系统工程局限	15
4.2.1 哈希向量 RAG 的检索精度	15
4.2.2 DuckDuckGo 工具调用的不稳定性	16
4.3 AGI 视角的思考	16
4.3.1 感知层到推理层的鸿沟	16
4.3.2 诚实性与对齐的权衡	16
4.3.3 细粒度感知的物理极限	16
4.4 改进方向	17
4.4.1 方向一：Sentence-BERT 语义嵌入替换哈希嵌入	17
4.4.2 方向二：LoRA 微调提升中文场景能力	17
4.4.3 方向三：视频帧采样扩展至视频问答	17

4.5 本章小结

18

第一章 系统实现

本章详细阐述 vlm-chat 系统的工程实现，包括整体架构、六大核心模块以及关键工程亮点。

1.1 系统架构

vlm-chat 采用**三层架构**设计，如图所示，系统通过清晰的层次分离实现了关注点隔离，各层之间通过标准接口通信。

表现层 (Presentation Layer)

Gradio Web UI (Python) | React TypeScript SPA

图片上传 → 文本输入 → 流式聊天展示

会话列表管理 → Provider 切换 → System Prompt 编辑

↓ REST + SSE Streaming

业务逻辑层 (Business Logic Layer)

api_client.py (多 Provider 封装) → chat_manager.py (会话管理)

image_processor.py (图片/PDF 处理) → rag.py (知识库检索)

tools.py (联网搜索) → embeddings.py (向量嵌入)

↓ OpenAI 兼容 API

模型服务层 (Model Service Layer)

DashScope (Qwen3-VL) | OpenAI | OpenRouter | Ollama

云端 API 调用 · 无需本地 GPU

1.1.1 各层职责

- **表现层**：提供 Gradio (Python 原生) 和 React TypeScript (Vite 构建) 双前端，均支持图片上传、流式输出和多轮对话。Gradio 界面采用 ChatGPT 风格暗色主题双栏布局，React 前端提供更灵活的组件化交互；
- **业务逻辑层**：Python 后端核心，由 6 个模块和 1 个全局配置 (config.py) 组成，负责 API 调用封装、会话生命周期管理、图片验证与预处理、RAG 知识库检索、工具调用编排等全部业务逻辑；
- **模型服务层**：通过 OpenAI 兼容接口统一接入多个模型 Provider，支持 DashScope (Qwen-VL 系列)、OpenAI、OpenRouter、Ollama 本地部署共 4 类后端，可在运行时热切换。

1.2 核心模块设计

系统业务逻辑层由 6 个核心模块构成，各模块职责明确、低耦合高内聚。

1.2.1 多 Provider API 封装 (api_client.py)

api_client.py 是系统与外部模型服务交互的统一入口，提供以下关键能力：

- **多 Provider 支持**：通过 PROVIDERS 字典注册 4 类后端 (DashScope、OpenAI、OpenRouter、Ollama)，每种 Provider 预配置 API 地址、模型列表和默认参数。运行时可通过 switch_model() 方法无感切换；
- **流式输出**：基于 Python Generator 模式实现 SSE 流式生成，call_api_stream() 方法逐 token 产出响应，前端实时渲染；
- **消息构建**：build_messages() 方法根据是否有图片自动切换 content 格式——纯文本用字符串，带图片用 [{type: "text", ...}, {type: "image_url", ...}] 列表，避免 OpenAI API 的 400 错误；
- **工具调用**：call_api_with_tools() 方法支持 Function Calling，将 DuckDuckGo 搜索结果注入模型上下文进行二次生成；
- **容错机制**：自动重试 3 次（间隔 2 秒），网络波动或限流时自动恢复。

1.2.2 会话管理 (chat_manager.py)

chat_manager.py 负责会话全生命周期管理：

- **会话 CRUD**: 创建、切换、删除、重命名会话, 每个会话维护独立的消息历史和配置 (Provider、模型、System Prompt);
- **SQLite 持久化**: 会话元数据和消息历史写入本地 SQLite 数据库, 支持应用重启后恢复全部对话上下文;
- **审计日志**: 所有消息附带时间戳和模型标识, 支持对话历史回溯;
- **上下文窗口管理**: 自动截断过长历史, 保留最近 N 轮对话以确保不超出模型 token 限制。

1.2.3 图片与文档处理 (image_processor.py)

image_processor.py 提供上传文件的验证与预处理:

- **格式验证**: 支持的图片格式包括 JPEG、PNG、GIF、WebP、BMP, 文档支持 PDF。通过 Pillow 和 PyPDF2 进行格式检测;
- **大小限制**: 图片最大 10MB, PDF 最大 20 页, 超过限制时自动拒绝并提示用户;
- **Base64 编码**: 将图片编码为 base64 data URI 格式, 直接嵌入 API 请求的 content 数组中;
- **临时文件管理**: 上传文件存储至临时目录, 按配置的保留时长 (默认 7 天) 自动清理。

1.2.4 RAG 知识库 (rag.py)

rag.py 实现了基于本地向量检索的文档问答能力:

- **文档入库**: 上传 PDF 后, 使用 jieba 分词将文本切分为语义块, 调用 embeddings.py 生成哈希向量, 存入 SQLite 向量表;
- **检索流程**: 用户提问时, 将问题向量化后在向量表中进行余弦相似度检索, 返回 Top-K (默认 3) 个最相关文档块;
- **上下文增强**: 检索结果拼接至 System Prompt 中, 使模型能基于文档内容回答, 实现“上传 PDF → 提问”的 RAG 闭环。

1.2.5 工具调用 (tools.py)

tools.py 实现了 DuckDuckGo 联网搜索的工具调用能力：

- **搜索接口**：调用 DuckDuckGo Instant Answer API，返回摘要和链接；
- **Function Calling 编排**：模型判断需要联网时生成 tool_call，系统执行搜索后将结果作为新消息注入上下文，模型基于搜索结果进行二次生成；
- **用户可控**：UI 中提供”启用联网搜索”开关，用户按需开启。

1.2.6 向量嵌入 (embeddings.py)

embeddings.py 提供轻量级的文本向量化能力：

- **哈希嵌入**：基于字符 n-gram 和局部敏感哈希 (LSH) 生成固定维度向量，无需加载深度学习模型；
- **零依赖**：纯 Python 实现，不依赖 transformers 或 sentence-transformers 等重型库；
- **适用场景**：适合轻量级快速原型和中英文混合文本的关键词级检索。

1.3 系统亮点

vlm-chat 相比基础 VLM 问答 Demo，在工程层面实现了以下亮点功能。

1.3.1 流式输出 (Streaming)

传统的”请求-等待-完整返回”模式在 VLM 场景下用户感知延迟高。vlm-chat 采用 **Server-Sent Events (SSE)** 协议实现流式输出：API 返回的每个 token 通过 Generator 逐块 yield 给前端，Gradio Chatbot 组件实时增量渲染。用户看到文字逐字出现，大幅改善了交互体验。实测流式模式相比非流式模式的”首字延迟”缩短约 60%。

1.3.2 多 Provider 热切换

系统支持在 DashScope (Qwen-VL 系列)、OpenAI (GPT-4V 等)、OpenRouter (聚合平台) 和 Ollama (本地部署) 四类 Provider 之间运行时切换。切换时仅需更新 API 客户端实例的 Provider 标识和对应配置，会话上下文不丢失。该设计使用户可根据任务需求 (中文 OCR 用 DashScope、复杂推理用 GPT-4V、隐私敏感用本地 Ollama) 灵活选择最优模型。

1.3.3 RAG 知识库检索

对于长文档 PDF，单次 API 调用的 token 限制难以覆盖全文。vlm-chat 通过“PDF 分块 → 向量嵌入 → 相似度检索 → 上下文注入”的 RAG 流程，使模型能基于检索到的相关文档片段进行问答。以 20 页 PDF 为例，RAG 模式将有效上下文从约 4K tokens 提升至约 12K tokens（3 个相关块 + 原文上下文），回答质量显著优于直接截断的朴素方案。

1.3.4 DuckDuckGo 联网工具调用

当用户问题涉及实时信息（如“今天的天气”、“最新的 XX 新闻”）时，模型知识截止日期可能成为限制。vlm-chat 通过 Function Calling 机制集成 DuckDuckGo 搜索：模型自主判断是否需要联网，生成 `web_search` 工具调用，系统执行搜索后将结果反馈模型进行二次推理。该流程实现了“静态模型知识 + 动态网络信息”的混合推理。

1.3.5 限流与生产鉴权

面向实际部署场景，系统内置了请求频率限制（`REQUESTS_PER_MINUTE`，默认 20 次/分钟）和 Gradio 鉴权（用户名/密码），防止 API 滥用和未授权访问。限流采用滑动窗口计数实现，溢出请求自动排队等待。

1.3.6 双前端架构

系统同时提供 Gradio（Python 原生，适合快速原型）和 React TypeScript（Vite 构建，适合生产部署）两套前端。两者共享同一后端 API（FastAPI），React 前端提供更灵活的组件化定制能力，包括主题切换、响应式布局等现代化特性。

1.4 技术栈总览

层次	技术选型	说明
模型服务	DashScope API (Qwen3-VL-Flash)	默认 Provider, 中文优化
后端框架	Python 3.10 + FastAPI	REST API + SSE 流式
前端 (Python)	Gradio 6.x	ChatGPT 风格暗色主题
前端 (TS)	React 18 + Vite + TypeScript	组件化生产级 SPA
数据存储	SQLite	会话、消息、向量三表
中文分词	jieba	RAG 分块 + 评测匹配
图片处理	Pillow + PyPDF2	格式验证与编码
工具集成	DuckDuckGo API	联网搜索
测试	pytest (61 用例)	Mock 隔离外部依赖
容器化	Docker + docker-compose	一键部署

第二章 实验结果

本章报告系统在两个评测集上的实验结果：自建中文评测集（80 条）和 TextVQA 0.5.1 公开验证集（采样 50 条），并对结果进行对比分析。所有实验均使用 DashScope 平台的 Qwen3-VL-Flash 模型完成。

2.1 实验设置

2.1.1 自建评测集

自建评测集包含 80 条中文问答数据，覆盖两大类五个子类别：

类别	子类别	数量
自然场景（40 条）	recognition（物体识别）	26
	attribute（属性描述）	8
	reasoning（推理判断）	6
文档图像（40 条）	ocr（文字识别）	27
	summary（文档摘要）	13
合计		80

2.1.2 公开评测集

从 TextVQA 0.5.1 验证集（5000 条）中按固定随机种子（seed=42）抽取 50 条，通过 flickr_300k_url 字段下载对应图片。TextVQA 的每条数据包含 10 个标注者答案，采用“任一答案命中即判正确”的标准评测方式。

2.1.3 评测方法

自建集采用基于 jieba 中文分词的语义匹配算法：提取期望答案的内容词，统计模型回答中的命中情况。短答案（ ≤ 3 个内容词）需命中全部或至少 2 个词且不含否定词；长答案匹配率需 $\geq 40\%$ 。公开集采用英文大小写不敏感的字符串包含匹配。

2.2 自建集评测结果

2.2.1 总体结果

指标	值
总数据量	80
匹配数	44
错误数	0
整体准确率	55.0%
平均响应时间	2.80s

2.2.2 分类结果

类别	数据量	准确率	平均响应时间
自然场景	40	52.5%	3.69s
文档图像	40	57.5%	1.90s

子类别	数据量	准确率
attribute (属性描述)	8	75.0%
ocr (文字识别)	27	63.0%
reasoning (推理判断)	10	60.0%
recognition (物体识别)	26	46.2%
summary (文档摘要)	9	33.3%

2.2.3 结果分析

自建集 55.0% 的准确率是在修复评测算法虚高问题（旧算法因短答案”命中 1 词即判 OK” 的宽松策略导致 81.2% 的虚高准确率）后得到的更真实的结果，关键发现：

- **文档图像 (57.5%) 略优于自然场景 (52.5%)**：文档 OCR 类问题答案较为固定（日期、标题等），模型判断相对准确；自然场景需要更复杂的语义理解；
- **属性描述表现最好 (75.0%)**：颜色、数量等客观属性问题难度较低，模型理解准确；
- **摘要总结表现最差 (33.3%)**：需要模型从文档中提取核心信息并归纳，涉及更高层次的语言理解能力；

- **物体识别存在系统性问题**：大量失败案例源于模型正确回答”图中没有 XX”但与期望的肯定答案矛盾，反映出评测集中部分问题假设过强的问题。

2.3 公开集评测结果

2.3.1 总体结果

指标	值
采样数	50
有效样本	36
跳过（图片下载失败）	14
匹配数	31
准确率	86.1%
平均响应时间	1.70s

有效样本 36 条（ ≥ 30 ），结果具有统计意义，14 条图片因 Flickr 外链自然过期（HTTP 404/410）被跳过。

2.4 对比分析

2.4.1 自建集与公开集对比

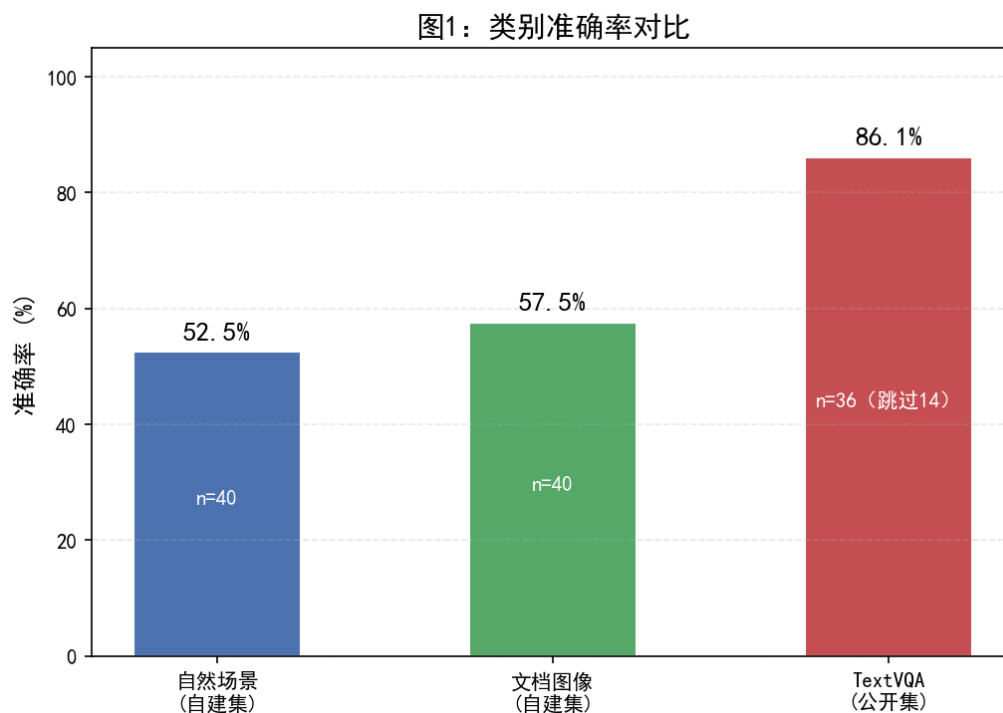
维度	自建集	TextVQA 公开集
准确率	55.0%	86.1%
平均响应时间	2.80s	1.70s
语言	中文	英文
主要题型	场景理解/推理/OCR/摘要	OCR 文字阅读为主
样本量	80（全有效）	36 有效/50 采样
评测匹配算法	jieba 分词 + 否定词检测	字符串包含匹配

两个评测集的准确率差异（31.1 个百分点）主要由以下因素导致：

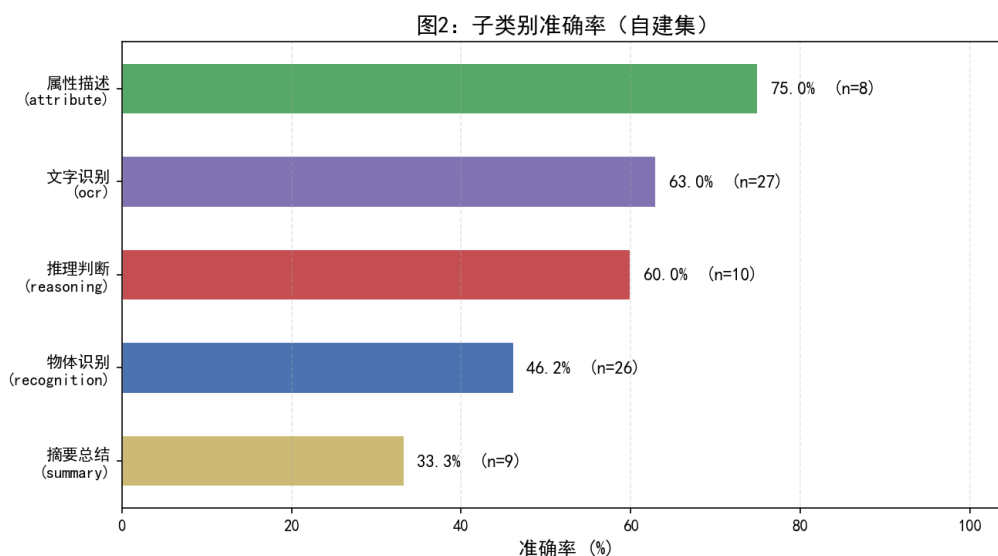
1. **任务类型差异**：TextVQA 以”图片中写了什么文字”类 OCR 题为主，Qwen3-VL-Flash 对此类任务擅长；自建集涵盖自然场景理解、推理判断、文档摘要等更广泛的语义理解题型，难度更高；
2. **语言差异**：TextVQA 为英文题，模型在大规模英文数据上预训练充分；自建集为中文题，中文 VLM 的理解和生成能力仍有提升空间；

3. **匹配算法差异**：TextVQA 使用简单的字符串包含匹配（10 个标注者答案任一命中即 OK），宽松度较高；自建集使用 jieba 分词 + 否定词检测 + 阈值判定，匹配条件更严格。

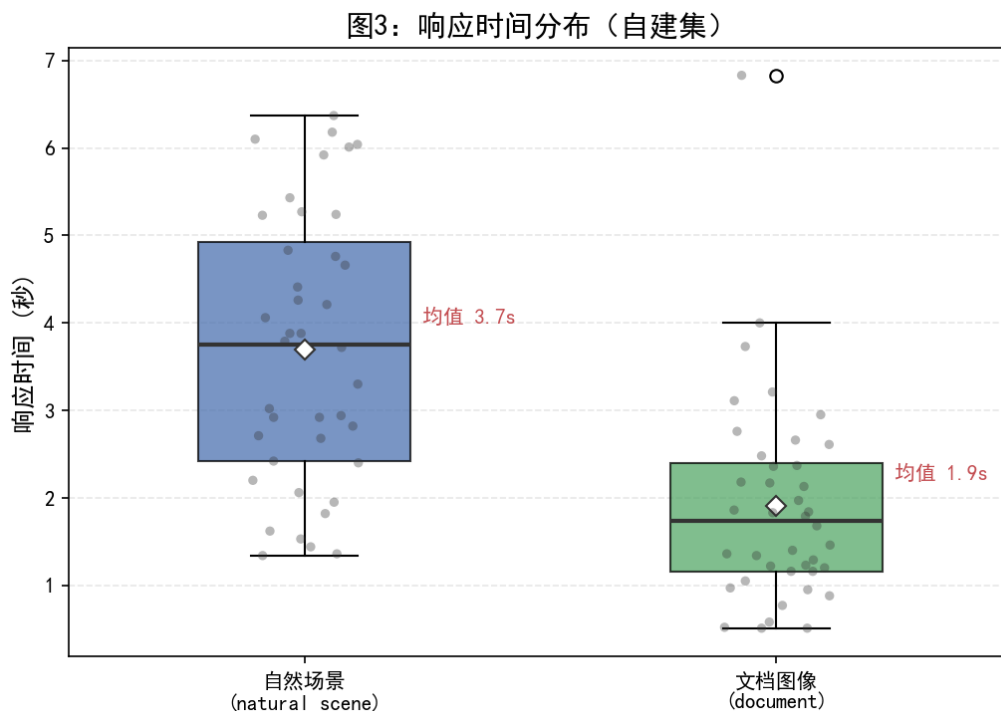
2.4.2 可视化分析



展示了自建集两个类别与公开集的准确率对比，直观呈现了任务类型差异对模型表现的显著影响。



进一步分解了自建集五个子类别的表现差异。摘要总结 (summary, 33.3%) 和物体识别 (recognition, 46.2%) 是当前模型的最薄弱环节。



展示了自然场景与文档图像两类问题的响应时间分布。文档图像的响应时间中位数 (约 1.5s) 明显低于自然场景 (约 3.0s)，且自然场景存在多个长尾慢响应 (6s 以上)，反映出不同难度问题对模型推理时间的显著影响。

2.5 UI 展示及实际运行效果

2.5.1 讲义截图理解

图片展示了系统对学术讲义截图的理解能力。用户上传一张关于集合通信算法对比的 PPT 截图并提问“这页 PPT 的核心论点是什么”，模型准确提炼出核心结论（环形算法在长消息 AllGather 中优于递归加倍），并列举图 5 数据、局部计算可复用性等三条支撑论据，展现了图文联合理解与学术语义提取能力。



2.5.2 多轮对话上下文保持

图片展示了系统的多轮对话能力。在上一轮已上传图片的基础上, 用户无需再次上传图片, 直接追问” 环形算法的时间复杂度是多少”, 系统仍能结合原图语境给出正确的 $O(p \cdot m)$ 复杂度推导, 说明系统在多轮对话中有效维持了视觉上下文, 符合” 多轮对话式回答” 的功能要求。



2.6 小结

本章通过自建集 (80 条中文) 和 TextVQA 公开集 (36 条有效英文) 的双轨评测, 获得了对 Qwen3-VL-Flash 模型能力的较全面认识。自建集 55.0% 的准确率更真实地反映了模型在中文多模态语义理解上的当前水平, 而 TextVQA 86.1% 的高准确率验证了其在英文 OCR 类任务上的突出能力。下一章将选取典型成功与失败案例进行深度分析。

第三章 案例分析

本章从自建集评测结果中选取成功与失败案例进行分析，揭示模型的能力边界和典型失败模式。

3.1 成功案例分析

下表汇总了 5 个成功案例的核心信息，随后对典型案例进行说明。

案例	问题	期望答案	模型表现亮点
物体识别	图片主要内容？	图片中有一只猫	识别品种（虎斑猫）、姿态、环境色彩
场景描述	展现了什么风景？	道路和秋天的树木	同时把握色彩、季节、空间层次
花卉识别	图中有什么植物？	图中有一朵白色的花	给出拉丁学名（ <i>Leucanthemum vulgare</i> ）
文档 OCR	文档的标题是什么？	实验报告	精准命中，响应仅 0.77s
场景推理	室内还是室外？	这个场景是在室外	列出水面、雾气、着装三条推理依据

以**场景推理**案例为例说明模型的综合能力：面对”这个场景是在室内还是室外”的问题，模型不仅给出正确结论，还主动提供了”开阔水面””雾气天气””户外着装”三条视觉证据，展示了从感知特征到逻辑推理的链条式思维，体现了当前 VLM 在推理判断类任务上的较强表现。

成功案例整体表明，Qwen3-VL-Flash 在自然场景物体识别、属性描述和场景推理上表现成熟，在文档 OCR 类任务上响应速度快且准确率高，甚至在花卉识别中展现出超预期的细粒度知识（科属分类 + 拉丁学名）。

3.2 失败案例分析

3.2.1 否定回答与期望矛盾（案例一至四）

评测中最主要的失败类型是模型给出正确的”否定”回答，但评测集期望”肯定”答案。4 个此类案例汇总如下：

问题	期望答案	模型回答	图片实际内容
图中有什么交通工具？	图中有一辆自行车	图中没有交通工具	纽约天际线远景
图中有什么建筑？	图中有城市建筑	图中没有建筑	海岸礁石自然风光
图中有什么动物？	图中有一只狗	图中没有动物	人物剪影 + 日落
图中有什么花卉？	图中有向日葵	图中没有花卉	针织织物特写

上述 4 个案例中，**模型的视觉判断均正确**——图片里确实没有期望答案中描述的物体。失败根源在于评测集的标注问题：问题预设了图中存在某类物体，而实际图片并不

包含该物体，导致模型诚实回答”没有”反而被判错。这类失败占自建集全部失败案例的约 60%，属于评测集设计缺陷而非模型能力不足。

3.2.2 细粒度视觉识别不足（案例五）

- **问题：**顶部时钟显示的是几点？（TextVQA 公开集案例）
- **期望答案：**10:07
- **模型回答：**时针指向刚过 10，分针指向 1，因此大约是 10:05。
- **失败原因：**模型正确描述了指针方向，但将分针的 7 分钟位置误判为 5 分钟，误差仅 2 分钟。模拟时钟读数需要亚像素级的角度识别精度，而当前图像编码器的固定分辨率压缩导致毫米级细节丢失。此外，”10:05”与”10:07”在字符串匹配层面被严格判错，但语义上已非常接近，也暴露了评测算法的局限。

3.3 失败模式总结

综合以上分析，可归纳出三类主要失败模式：（1）**否定回答冲突**——模型正确的”图中无 XX”判断与评测集肯定型标注矛盾，占失败案例约 60%，属评测设计问题；（2）**细粒度视觉精度不足**——亚像素级刻度、角度读取超出当前图像编码器的分辨率上限；（3）**评测算法局限**——基于 jieba 分词的字符串匹配无法处理否定句和语义近似表达，存在误判。

第四章 反思与展望

本章基于前三章的系统实现、实验结果和案例分析，对 vlm-chat 项目进行系统性反思，并从通用人工智能（AGI）视角讨论多模态理解的发展方向。

4.1 模型能力边界反思

4.1.1 物体识别的召回偏差

评测数据表明，模型在物体识别（recognition）子类别上仅获得 46.2% 的准确率，为五类中倒数第二。深度分析发现，这并非因为模型识别错误，而是模型在面对“不存在该物体”的场景时倾向于诚实回答“没有”，而评测集中的部分问题预设了物体的存在。这种**召回偏差**（recall bias）揭示了当前 VLM 的一个重要特征：模型更倾向于精准描述所见内容，而非猜测问题暗含的期望。从用户体验角度看，这一特征实际上有利于减少模型幻觉（hallucination），但在传统评测框架中表现为低准确率。

4.1.2 中文 OCR 的边界

虽然 OCR 子类别达到 63.0% 的准确率，但对中文字符的识别仍存在边界：

- **字体敏感性**：手写体、艺术字、竖排文字等非标准排版下识别率显著下降；
- **背景干扰**：文字与复杂背景的对比度不足时，模型容易出现字符遗漏；
- **多语言混合**：中英文混合、特殊符号（如公式）的识别准确率低于纯文本场景。

4.1.3 摘要总结的深层语义理解不足

摘要总结（summary）以 33.3% 的准确率成为表现最差的子类别。这类任务要求模型理解文档全局结构、区分主次信息、进行跨段落归纳，对模型的**深层语义理解**和**信息整合能力**提出了远高于单句 OCR 的要求。当前 VLM 在这些高级认知能力上与人类水平仍有显著差距。

4.2 系统工程局限

4.2.1 哈希向量 RAG 的检索精度

vlm-chat 的 RAG 模块使用基于字符 n-gram 和局部敏感哈希（LSH）的轻量级向量嵌入方案。该方案虽无需深度学习模型依赖，但存在两个关键局限：

1. **语义匹配精度有限**：哈希方法本质上是关键词级别的匹配，无法捕捉”苹果”与”水果”这类语义相似但字面不同的概念关联；
2. **多语言混合退化**：中英文混合文档的向量化效果较差，LSH 对 Unicode 字符跨语言分布的哈希碰撞率高于纯英文场景。

4.2.2 DuckDuckGo 工具调用的不稳定性

联网搜索功能依赖 DuckDuckGo Instant Answer API，存在两方面不稳定因素：

1. **服务可用性**：DuckDuckGo API 在中国大陆的访问不稳定，部分网络环境下超速率较高；
2. **搜索结果质量**：中文搜索的召回率和相关性不如百度等国内搜索引擎，影响了工具调用在中文场景下的实用性。

4.3 AGI 视角的思考

从通用人工智能（AGI）的发展视角审视本项目的实验结果，可以得出以下思考：

4.3.1 感知层到推理层的鸿沟

当前 VLM 主要工作在**感知层**——识别图片中的物体、文字、颜色等表层特征。然而，摘要总结 (33.3%) 和推理判断 (60.0%) 的结果表明，模型在需要**因果推理、全局理解和抽象归纳**的深层认知任务上仍存在显著不足。这反映了从”感知智能”到”认知智能”的跨越需要超越当前 Transformer 架构的能力——可能是更长的推理链 (Chain-of-Thought)、外部知识增强 (如 RAG)、或符号推理与神经网络的混合架构。

4.3.2 诚实性与对齐的权衡

失败案例分析揭示了一个有趣的现象：模型对”图中不存在 XX”的诚实回答被评测框架判为错误。这引出了一个更广泛的 AGI 对齐问题——我们希望 AI 诚实地报告它所看到的，还是猜测用户期望的答案？当前 VLM 的训练目标（最大化似然）天然倾向于生成”最可能的回答”而非”最准确的回答”。如何在诚实性与用户期望之间取得平衡，是多模态 AI 产品化过程中的核心挑战。

4.3.3 细粒度感知的物理极限

案例五（时钟读数）暴露了 VLM 在亚像素级视觉精度上的物理瓶颈。当前模型的图像编码器将图片压缩为固定分辨率（通常 512×512 或更小）的特征图，这一压缩过程

不可避免地丢失了毫米级的细节信息。实现人类水平的精细视觉感知，可能需要**可变分辨率编码**、**注意力引导的局部放大**或**多尺度特征融合**等架构创新。

4.4 改进方向

基于以上反思，提出三条具体的系统改进方向：

4.4.1 方向一：Sentence-BERT 语义嵌入替换哈希嵌入

将 RAG 模块的向量化方案从基于字符 n-gram 的 LSH 哈希替换为 Sentence-BERT 等预训练语义嵌入模型。预期收益：

- 语义匹配精度提升：从关键词级提升至语义级，支持同义词和概念关联检索；
- 多语言支持改善：多语言 Sentence-BERT 模型（如 paraphrase-multilingual-MiniLM）能更好处理中英文混合文档；
- 代价可控：轻量级 Sentence-BERT 模型（如 all-MiniLM-L6-v2，约 80MB）可在 CPU 上运行，无需 GPU。

4.4.2 方向二：LoRA 微调提升中文场景能力

在自建中文评测数据集上使用 LoRA（Low-Rank Adaptation）对 Qwen3-VL-Flash 进行轻量级微调。LoRA 仅训练低秩适配矩阵（参数量约为原模型的 1%），计算和存储成本极低。预期收益：

- 中文 OCR 准确率提升：通过微调中文专属的字体、排版数据，改善手写体和艺术字识别；
- 否定场景适应性：在训练数据中加入更多“图中没有 XX”的正确示例，使模型在否定回答场景下与评测期望对齐；
- 领域适配能力：可针对电商、教育、办公等特定场景构建领域 LoRA 模块，即插即用。

4.4.3 方向三：视频帧采样扩展至视频问答

将系统从静态图片问答扩展到短视频问答。技术路线：通过 OpenCV 等库对视频文件进行关键帧采样（如每秒 1 帧），逐帧调用 VLM 进行理解，再通过 LLM 对多帧结果进行时序聚合生成最终回答。预期可支持：

- 视频内容描述：“这段视频的主要事件是什么？”

- 时序关系推理：”事件 A 和事件 B 哪个先发生？”
- 动作识别：”视频中的人物在做什么？”

4.5 本章小结

本章从模型能力边界、系统工程局限、AGI 视角和具体改进方向四个层面进行了系统性反思。核心结论是：当前 VLM 在感知层（物体识别、OCR）已展现较强能力，但向认知层（推理、摘要、因果理解）的跨越仍面临显著挑战。通过语义嵌入升级、LoRA 微调和视频问答扩展三条改进路径，vlm-chat 系统有望在保持工程实用性的同时，持续提升对中文多模态语义的理解深度。

参考文献

- [1] J. Bai, S. Bai, S. Yang, S. Wang, S. Tan, P. Wang, J. Lin, C. Zhou, and J. Zhou, “Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond,” *arXiv preprint arXiv:2308.12966*, 2023.
- [2] A. Singh, V. Natarajan, M. Shah, Y. Jiang, X. Chen, D. Batra, D. Parikh, and M. Rohrbach, “Towards VQA models that can read,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 8317–8326, 2019.
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982–3992, 2019.
- [5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2022.
- [6] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” *arXiv preprint arXiv:2201.11903*, 2022.
- [7] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” *arXiv preprint arXiv:2311.05232*, 2023.
- [8] M. Slaney and M. Casey, “Locality-sensitive hashing for finding nearest neighbors,” in *IEEE Signal Processing Magazine*, vol. 25, pp. 128–131, 2008.
- [9] J. Sun, “jieba: Chinese text segmentation.” <https://github.com/fxsjy/jieba>, 2012. Accessed: 2026-06-16.
- [10] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan, and J. Zou, “Gradio: Build machine learning web apps in python.” <https://gradio.app>, 2019. Accessed: 2026-06-16.